

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1408

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

CLASS TEST

Code of Conduct:

*I Ranjitha R (192421285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Q. No	Understanding of Concepts (25)	Explanation (30)	Application (20)	Organization(15)	Accuracy(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Annexure A
CLASS TEST

Q1. Write down the translation scheme to generate code for assignment statement. List the scheme for generating three address code for the assignment statement $x = (a + b) * (c + d)$ (10 Marks)

Q2. Consider the following Boolean expression:

$(a > b) \ \&\& \ (c < d) \ || \ (e < f)$

The grammar for Boolean expressions is given as:

$B \rightarrow B1 \ || \ B2$

$B \rightarrow B1 \ \&\& \ B2$

$B \rightarrow ! \ B1$

$B \rightarrow E1 \ \text{relop} \ E2$

$B \rightarrow \text{true}$

$B \rightarrow \text{false}$

(i). Explain how operator precedence and associativity affect the evaluation of the given Boolean expression.

(ii). Construct the Three Address Code (TAC) for the given Boolean expression using short-circuit evaluation.

(iii). Represent the control flow of the Boolean expression using appropriate labels and conditional jump statements. (10 Marks)

Q3. Given the declaration sequence

`int a, b[10];`

`float c, d;`

derive the intermediate representation including symbol table entries, type sizes, and offset calculations. (10 Marks)

Q4. Consider the following program fragments:

(i). while (A < C && B > D) do

 if (A == 1) then

 C = C + 1

 else

 while (A <= D) do

 A = A + B

Generate the Three Address Code (TAC) for the above program fragment. Clearly show:

(a) Conditional and unconditional jump statements

(b) Use of labels for control flow

(c) Evaluation of Boolean expressions (05 Marks)

(ii). main()

{

 int i;

 int a[10];

 i = 1;

 while (i <= 10)

 {

 a[i] = 0;

 i = i + 1;

 }

}

Translate the executable statements of the above C program into Three Address Code (TAC).

Clearly indicate:

(a) Representation of array elements

(b) Loop control using labels

(c) Increment and assignment operations (05 Marks)

Q5. Use backpatching to generate intermediate code for:

 if (a < b && c < d)

 x = a + b;

else
 $x = c + d;$

Show the intermediate instructions before and after backpatching. (10 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps

Co-4

ASSESSMENT_1

NAME : RANJITHA R

REG No: 192421285

COURSE CODE : CSA1408

COURSE NAME: COMPILER DESIGN



1. Translation scheme for Assignment statement:

Given,

$$x = (a+b) * (c+d)$$

Translation scheme:

$$S \rightarrow id = E$$

The translation rule is:

$$S \rightarrow id = E$$

$$\{ \text{emit}(id, \text{lexeme} = E, \text{place}); \}$$

For expression:

$$E \rightarrow E_1 + E_2$$

$$\{ E.\text{place} = \text{newtemp}(); \}$$

$$\text{emit}(E.\text{place} = E_1.\text{place} + E_2.\text{place}); \}$$

$$E \rightarrow E_1 * E_2$$

$$\{ E.\text{place} = \text{newtemp}(); \}$$

$$\text{emit}(E.\text{place} = E_1.\text{place} * E_2.\text{place}); \}$$

$$E \rightarrow (E_1)$$

$$\{ E_1.\text{place} = E.\text{place}; \}$$

$$E \rightarrow id$$

$$\{ E.\text{place} = id.\text{lexeme}; \}$$

Three Address code:

$$x = (a+b) * (c+d)$$

$$t_1 = a + b$$

$$t_2 = c + d$$

$$t_3 = t_1 * t_2$$

$$x = t_3$$

2) Boolean Expression :-

$$(a > b) \&\& (c < d) \parallel (e < f)$$

i) operator precedence and associativity:

The precedence of the operators is:

Relational operators : $>, <$

↓

Logical AND : $\&\&$

↓

Logical OR : $\parallel$

Therefore, the expression is evaluated as:

$$((a > b) \&\& (c < d)) \parallel (e < f)$$

Both $\&\&$ and $\parallel$ are left associative, but $\&\&$ has higher precedence than $\parallel$.

* $\&\&$: If $(a > b)$ is false, $(c < d)$ is not evaluated

* $\parallel$: If $((a > b) \&\& (c < d))$ is true, $(e < f)$ is not evaluated.

ii) TAC using short-circuit evaluation :-

use labels:

L_1 = true result

L_2 = false result

$L_3 \rightarrow L_4, L_5$ = intermediate labels.

TAC:

if $a > b$ goto L_1

goto L_3

L_1 : if $c < d$ goto L_2
goto L_3

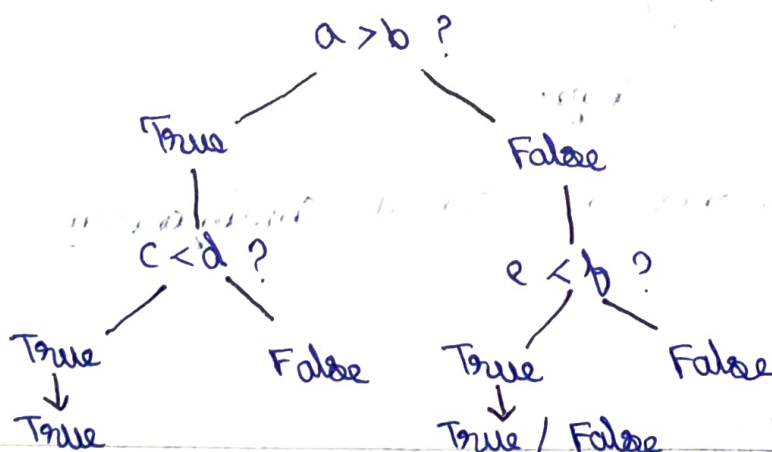
L_3 : if $e < d$ goto L_2
goto L_4

L_2 : result = true
goto L_5

L_4 : result = false

L_5 :

iii) control flow representation :-



More precisely:

L0 : if $a > b$ goto L1
goto L3

L1 : if $c < d$ goto L2
goto L3

L3 : if $e < f$ goto L2
goto L4

L2 : result = true
goto L5

L4 : result = false

L5 :

3) Intermediate Representation:

Given:

int a, b [10];

float c, d;

Assume :-

int size = 4 bytes

float size = 4 bytes

Array elements are stored contiguously

Type Information :-

int $\rightarrow$ 4 bytes:

float $\rightarrow$ 4 bytes

b[10] $\rightarrow 10 \times 4 = 40$ bytes

Symbol table:

Name	Type	width	Offset
a	int	4 bytes	0
b	int[10]	40 bytes	4
c	float	4 bytes	44
d	float	4 bytes	48

Total storage : $4 + 40 + 4 + 4 = 52$ bytes

Offset Calculation :-

For a:

$$\text{Offset (a)} = 0$$

For b:

$$\begin{aligned}\text{Offset (b)} &= \text{Offset (a)} + \text{size (int)} \\ &= 0 + 4 = 4\end{aligned}$$

For c:

$$\begin{aligned}\text{Offset (c)} &= \text{Offset (b)} + \text{size (b)} \\ &= 4 + 40 = 44\end{aligned}$$

For d:

$$\begin{aligned}\text{Offset (d)} &= \text{offset (c)} + \text{size (float)} \\ &= 44 + 4 = 48\end{aligned}$$

Thus, the intermediate representation contains the symbol information and memory layout.

a: int [offset 0, width 4]

b: int [10] [offset 4, width 40]

c: float [offset 44, width 4]

d: float [offset 48, width 4]

4. i) while ((A < c) and (B > D)) do

if A = 1 then C = C + 1

else while AC = D do

A = A + B

Three Address Code:-

100 : if A < c then goto 102

101 : goto 115

102 : if B > D then goto 104

103 : goto 115

104 : if A = 1 then goto 106

105 : goto 109

106 : $t_1 = C + 1$

107 : $C = t_1$

108 : goto 100

109 : if $A \leq D$ then goto 111

110 : goto 100

111 : $t_2 = A + B$

112 : $A = t_2$

113 : goto 109

114 : goto 100

115 :

• Main ()

{

int i ;

int a[10] ;

i = 1 ;

while (i ≤ 10)

{ a[i] = 0 ;

i = i + 1 ;

}

}

Three Address Code:

100 : $i = 1$

101 : if $i < 10$ then goto 103

102 : goto 108

103 : $t_2 = i$

104 : $t_2 = 4 * t_1$

105 : $a[t_2] = 0$

106 : $i = i + 1$

107 : goto 101

108 : —

5) Backpatching:-

~~if~~ $(a < d \text{ \& } c < d)$

$x = a + b;$

~~else~~

$x = c + d;$

Intermediate code before backpatching:-

1 : if $a < b$ goto —

2 : goto —

3 : if $c < d$ goto —

4 : goto —

5 : $t_1 = a + b$

6 : $x = t_1$

7 : goto —

8 : $t_2 = c + d$

9 : $x = t_2$

10 : —

Backpatch List:

For $a < b$:

true list = {1}

false list = {2}

Code After backpatching:-

1: If $a < b$ goto 3

2: goto 8

3: If $c < d$ goto 5

4: goto 8

5: $t_1 = a + b$

6: $x = t_1$

7: goto 10

8: $t_2 = c + d$

9: $x = t_2$

10:

Final Control Flow:-

